Lazy or Eager? When Post-Copy Process Restore Pays over a Network, and Streaming the Working Set

Utkarsh Maurya
Independent Researcher
utkarsh@kernex.sbs

***Abstract*—Restoring a checkpointed process on another machine can move the whole image first (eager) or start at once and fetch pages on fault (lazy, post-copy). Lazy restore is usually evaluated against an eager baseline that reads its image from local disk, which hides the cost the network adds to eager. We measure both honestly for CRIU process restore, with time to first response (TTFR) as the metric, over a grid of round-trip times (0–2 ms) and bandwidths from 100 Mb/s to 10 Gb/s, for a PyTorch inference server, a Redis-compatible key-value store and a Python web application. No fetch policy wins everywhere: demand paging wins only when bandwidth is scarce and RTT is near zero, prefetching wins in a narrow middle, and moving the whole image wins once bandwidth × RTT is large, which for the two random-access workloads is 9–10 of 12 grid points. Each lazy policy pays roughly one RTT per demand fault, and that term decides the map. We then remove it: the page server pushes a recorded first-touch trace over its own connection at line rate while demand faults keep a separate connection. With a trace from an earlier run this wins all 12 grid points for PyTorch and the web app (1.19–3.20× over the best existing policy) but only 6 of 12 for Redis, whose heap layout is not reproducible across runs on our x86 host. With a trace recorded on the same image it wins all 36 grid points (1.22–5.75×) and comes within 1–11 % of the working set's transfer time at 100 Mb/s and 1 Gb/s. On two Raspberry Pi 5s over a real 1 Gb/s link, streaming cuts TTFR by 1.8–4.8×. A trace alone, though, leaves every later request paging at one RTT per cold page; streaming the rest of the image behind the trace keeps a 2.3–2.5× faster first response and serves the following load sooner than eager restore does.**

I. Introduction

Checkpoint/restore moves running processes between machines: for load balancing, for draining a host, and increasingly for starting serverless functions and agent sandboxes from a warm snapshot. CRIU [1] offers two ways to restore on a remote host. Eager restore copies the image and then maps it. Lazy restore [2] maps the process at once with its memory registered with `userfaultfd` [3]; a handler fetches each page from a page server on the source the first time the process touches it.

The usual argument for lazy restore is that a process touches a small part of its memory before it can do useful work. That argument compares lazy restore over the network with eager restore from local disk. A real migration has no local image: eager restore must first move it, at image/bandwidth. Lazy restore pays one round trip per demand fault instead. Which cost is smaller depends on the image size, the working set, the bandwidth and the RTT, and no published study maps it for process restore. REAP [4], FaaSnap [5] and Spice [6] are evaluated on local storage; AWS records chunk access order for SnapStart and container loading [7], [8] but has not published a regime analysis.

This paper makes five contributions.

1) **A regime map** (§IV). We measure eager-over-network, demand paging, sequential/stride prefetch and an adaptive prefetcher across four RTTs and three bandwidths for three workloads. No policy wins everywhere. The losing policies are not slow because of bandwidth: each demand fault costs about one RTT, and the thousands of faults left after prefetching decide the outcome.
2) **Trace-ordered streaming** (§V), REAP's record-and-prefetch [4] adapted to a network. The page server pushes a recorded first-touch trace as one request over its own TCP connection, at whatever rate TCP achieves; the handler installs pages as they land. Demand faults keep their own connection. TTFR then depends on the working set's transfer time plus at least one RTT per page the trace missed.
3) **Where traces come from matters** (§VI, §VII). A trace from an earlier run of the same program suffices for PyTorch and the web app (96–99.9 % page overlap) but not for Redis on our x86 host (64.5 %), where allocator placement shifts whole pages between runs even with ASLR off. A trace recorded on the same image removes misses almost entirely and wins every grid point.
4) **After the first response** (§X). A trace covers only what the first request touched. Later requests page in at one RTT each, 50× below eager throughput at 2 ms. Streaming the rest of the image after the trace fixes this while keeping most of the TTFR gain.
5) **Real hardware** (§IX). On two Raspberry Pi 5s over a 1 Gb/s link the results hold, and one loopback effect, a large penalty for streaming the rest of the image through a shared FIFO, turns out to be an emulation artifact.

All measurements use `TCP_NODELAY` on every page-server socket. A companion report [9] shows that without it, Nagle's algorithm interacting with delayed ACKs inverts the measured effect of prefetching on this system; we take the corrected setup as given here.

II. Background and Related Work

Post-copy restore. Post-copy migration starts the target at once and pulls memory on demand. Hines et al. [10] combine demand paging with an active push of the remaining pages from the source and pre-paging around each fault,

for virtual machines on a gigabit LAN. Our stream-all and stream-pf are those two mechanisms with the push ordered by a recorded trace; what we add is when each pays off as RTT and bandwidth vary. CRIU implements post-copy for processes: `criu restore –lazy-pages` registers the restored memory with `userfaultfd` and hands the descriptor to a lazy-pages daemon, which resolves each fault with `UFFDIO_COPY`. An open CRIU pull request streams images for post-copy [11].

Snapshot cold starts. REAP [4] observed that serverless functions touch a stable working set after restore. It records that set once into a compact file and, on later restores, loads it with one read before the VM resumes; other pages fault on demand. FaaSnap [5] records the set from the host side and loads it concurrently with execution. stream-same is REAP's record-once policy with FaaSnap-style overlap, moved onto the network. Spice [6] shows that with the right kernel primitives local process restore approaches warm-start latency. These systems are evaluated restoring from local storage, where a page costs microseconds. REAP notes that with remote snapshot storage retrieval depends on the network's latency and bandwidth, but does not measure it. Over a network two further questions arise: whether a trace from another run still matches (a miss costs an RTT, §VII), and what the pages outside the trace cost after the first response (§X). MITOSIS [12] forks a process onto another machine and reads its parent's memory on demand with one-sided RDMA, at a few microseconds per page; we study commodity TCP at RTTs of 0.2–2 ms. Firecracker [13] documents the `userfaultfd` handling rules a snapshot handler must follow [14].

Network restore in industry. AWS Lambda loads container images and SnapStart snapshots on demand from a tiered cache and prefetches chunks in a recorded access order [7], [8]. That is the closest design to ours; the analysis of when it pays has not been published.

Far-memory prefetching. Leap [15] and 3PO [16] prefetch remote memory in steady state. Restore is a different problem: every page starts remote, and the access stream before the first response is short and mostly cold.

III. System and Methodology

Pipeline. The workload runs on the source, is checkpointed with CRIU, and is restored on the target. The page server is a Python TCP server that indexes the CRIU pagemap and serves 4 KiB pages by virtual address. The lazy-pages handler is a C daemon that owns the `userfaultfd`. It fetches the faulting page synchronously and can issue prefetches asynchronously off the fault path.

Policies. We compare:

- full: eager restore from images already on the target. This is a floor, not a contender: it moves no data.
- full-net: eager restore after streaming the whole image from the source over the same shaped link.
- lazy: demand paging only.
- prefetch: on each fault, also fetch the next pages sequentially, or along a detected stride.
- adaptive: prefetch with a controller that backs off when prefetched pages go unused.
- stream, stream-pf, stream-all, stream-same: trace-ordered streaming (§V).

Workloads. *PyTorch*: a ResNet-18 inference server; the probe writes a trigger file and waits for one inference result. Its x86 image is 80.4k pages (329 MB), of which 15.5k (63 MB, 19 %) are touched before the result. *Redis*: a key-value server holding 200k keys of 1 KiB; the probe issues a fixed set of 20k `MGET`s in batches of 100. On the x86 host the server binary is Valkey 9.1.2 (the distribution installs it as `redis-server`); on the Pis it is Redis 8.0.2. The x86 trace is 29.8k pages (122 MB). *Web app*: a Python `http.server` application over an in-memory SQLite database (4M orders, 50k customers, 20k products) that renders Jinja2 pages; each request runs an indexed query, aggregates the rows against cached Python objects and renders a template. The probe fetches 200 fixed customer pages. Its image is about 260 MB, of which 26.6k pages (109 MB, 42 %) are touched before the probe completes. Probes reach Redis and the web app over a Unix socket, so probe traffic is not shaped; the Pi kernel lacks `unix_diag`, so there they use TCP over loopback.

Metric. TTFR is the time from the start of `criu restore` until the probe completes: one inference, 20k keys, or 200 pages. For full-net it includes the image transfer.

Testbeds. *x86*: one AMD Ryzen 7 7735HS laptop, Linux 7.2.8, CRIU 4.2.1. Source and target share the host; the network is loopback shaped with `netem` [17]: delay d on `lo` (so RTT $= 2d$) and a rate cap. The grid is RTT $\in \{0, 0.2, 1, 2\}$ ms $\times$ rate $\in \{100\,\text{Mb/s}, 1\,\text{Gb/s}, 10\,\text{Gb/s}\}$. *Pi*: two Raspberry Pi 5s on a direct 1 GbE link (base RTT 154–165 μs). Pi A runs and restores the workload; Pi B runs the page server. `netem` delay goes on A's egress only, so B's data keeps B's real qdisc. Pi A runs Linux 6.18.59 with CRIU 4.2.1, except the first PyTorch run (CRIU 4.1.1, §IX).

Statistics. Five iterations per cell, each with a fresh workload and checkpoint. We report medians. On x86 the population standard deviation of the network policies is at most 3.6 % of the median; the local full floor (about 200 ms) reaches 6–10 % in three cells. On the Pi one web-app prefetch iteration at delay 0 finished in 339 ms with 2 faults, against about 2.5 s for the other four; it shows the same signature as the run we discarded in §X, and the median is unaffected. Streaming with a cross-run trace varies far more, up to 56 %, because the number of misses changes from run to run (§VII); stream-same reaches 30 % in one Redis cell. Every win we report holds for all five iterations, meaning the slowest iteration of the winning streaming mode beats the fastest iteration of the best existing policy, with two exceptions: one Redis cell we call a tie, and the x86 web app's cross-run win at 2 ms and 10 Gb/s, where two of five iterations (673 and 702 ms) lose to eager's fastest (575 ms). Demand-only lazy is skipped at 1 and 2 ms RTT on x86, where it exceeds the 30 s probe timeout.

IV. The Regime Map

Table I shows TTFR for the existing policies.

TABLE I

TTFR (MS, MEDIAN OF 5) FOR EXISTING POLICIES ON X86. BOLD: BEST PER WORKLOAD AND CELL. FULL (LOCAL IMAGES, NO TRANSFER) TAKES 172–253 MS THROUGHOUT AND IS OMITTED. IMAGE / PRE-RESPONSE WORKING SET: PYTORCH 329 / 63 MB, REDIS ABOUT 280 / 122 MB, WEB APP 260 / 109 MB. DATA: `REGIME/`, `REDIS-V2/`, `WEBAPP/`.

		PyTorch				Redis (Valkey)				Web app			
RTT	rate	full-net	lazy	prefetch	adaptive	full-net	lazy	prefetch	adaptive	full-net	lazy	prefetch	adaptive
0	100M	26664	**6278**	7706	7898	22311	**12331**	23202	23906	21253	**11153**	18022	19005
0	1G	2899	1388	**873**	889	**2427**	2687	2527	2515	2380	2607	**2184**	2202
0	10G	509	800	**335**	341	**440**	1550	744	756	**501**	1601	739	869
0.2	100M	26749	9405	**7855**	8060	22317	**18192**	24048	24656	21285	**16517**	19370	19874
0.2	1G	2904	4544	1153	**1134**	**2435**	8822	3520	3480	**2381**	8122	3669	3283
0.2	10G	**521**	3966	628	606	**436**	7803	1909	1825	**495**	7266	2506	2058
1	100M	26763	–	**9005**	9096	**22330**	–	28254	27257	**21278**	–	25342	23686
1	1G	2928	–	2550	**2286**	**2464**	–	8756	7775	**2412**	–	10140	7667
1	10G	**549**	–	2188	1711	**478**	–	7594	5779	**527**	–	9572	6662
2	100M	26765	–	10853	**10464**	**22354**	–	34679	31579	**21330**	–	33290	28985
2	1G	**2982**	–	4414	3726	**2496**	–	15704	12980	**2447**	–	19196	13679
2	10G	**627**	–	4231	3223	**533**	–	14800	10657	**579**	–	18832	13081

No single policy wins. For PyTorch there are three zones. Demand paging wins only at 100 Mb/s with zero RTT, where every extra byte is expensive and faults are cheap. Prefetch or the adaptive variant wins the middle of the grid. Eager full-net wins once bandwidth × RTT is large: at 10 Gb/s from 0.2 ms and at 1 Gb/s from 2 ms. For Redis and the web app the eager zone covers 10 and 9 of 12 cells; demand paging or prefetch wins only at zero or 0.2 ms RTT.

Prefetch overfetches and misses random access. For PyTorch, prefetch serves about 22k pages against a 15.5k-page working set (42 % extra), which is why it loses to demand paging at 100 Mb/s. Redis reads hash-table buckets and values at random, and the web app reads B-tree leaves and scattered Python objects; for both, sequential prefetch is worse than nothing at low RTT (Redis: 23.2 vs 12.3 s at 0/100 Mb/s; web app: 18.0 vs 11.2 s).

Each fault costs one RTT. PyTorch under demand paging takes 15.5k faults; raising the RTT from 0 to 0.2 ms adds 3.2 s, which is 15.5k times 0.2 ms. Prefetch leaves about 2k faults and gains 3.5 s from 0 to 2 ms RTT. Eager pays one bulk transfer and no per-fault RTT, which is why it wins at high RTT despite moving five times more data than the working set.

The gap. The cost a restore cannot avoid is moving the working set: for PyTorch, 63 MB is 5.1 s at 100 Mb/s, 0.51 s at 1 Gb/s and 51 ms at 10 Gb/s. The best existing policy is 1.24× above that at 100 Mb/s and zero RTT, and 5.9× above it at 1 Gb/s and 2 ms. A policy that moved exactly the working set, at line rate, without per-fault round trips, would close the gap.

V. TRACE-ORDERED STREAMING

Recording. A record run is a demand-only lazy restore in which the handler appends each fault address to a trace file, in fault order, until the first response. The trace is the working set before the first response, ordered by first touch. For traces reused across runs the workload is launched with ASLR off (`setarch -R`) so that addresses are comparable between runs.

Streaming. At restore the handler opens a second connection to the page server and sends one request: a magic marker, a count and the list of trace addresses. The server answers with the pages back to back, in trace order, without further requests. TCP alone sets the rate. A receive thread installs each page with `UFFDIO_COPY` as soon as its 4 KiB are complete and publishes how far into the trace it has got.

Demand path. Faults are served on the original connection, so a fault never waits behind the stream at the client. The handler classifies each fault against the stream: *landed* (the stream already delivered it; the fault raced the install), *ahead* (in the trace, not yet delivered) or *miss* (not in the trace). Ahead and miss pages are fetched on demand; an install that loses to the stream fails with `EEXIST` and is ignored.

Variants. stream uses a trace recorded on an earlier run. stream-pf adds sequential/stride prefetch, but only around misses. stream-all follows the trace with every other image page, ordered by distance to the nearest traced page. stream-same first runs an untimed record pass on the same image and then restores it again, streaming that trace; it models a system that records once per snapshot and restores it many times, as REAP does.

What it took. A first version reused the prefetch thread: one 8 B request per page, up to 1024 in flight. The Python server then became the bottleneck, and at 2 ms RTT the request window let the process overtake the stream, so most faults were "ahead". Server push removed both: the server reaches about 275k pages/s on loopback at 10 Gb/s and line rate at 1 Gb/s, and the 0 ms/10 Gb/s PyTorch cell fell from 405 to 129 ms.

VI. STREAMING RESULTS ON X86

Table II and Fig. 1 summarize the streaming modes.

Cross-run traces. For PyTorch a stream mode wins all 12 cells, by 1.19–3.20× over the best existing policy; plain stream is best at 100 Mb/s and at low RTT, stream-pf at 0.2 ms/10 Gb/s and at 1–2 ms/1 Gb/s, and stream-all at 10 Gb/s with RTT ≥ 1 ms. TTFR still rises with RTT (585 to 1028 ms at 1 Gb/s) because a few hundred faults miss the trace. The

TABLE II

STREAMING VS THE BEST EXISTING POLICY ON X86 (TTFR, MS, MEDIAN OF 5). "OLD": BEST OF TABLE I. "CROSS": BEST OF STREAM, STREAM-PF, STREAM-ALL WITH A TRACE FROM AN EARLIER RUN. "SAME": STREAM-SAME. ×: SAME OVER OLD. DATA: `REGIME-STREAM/`, `PYTORCH-SAME/`, `REDIS-V2/`, `REDIS-SAME/`, `WEBAPP/`.

		PyTorch				Redis				Web app			
RTT	rate	old	cross	same	×	old	cross	same	×	old	cross	same	×
0	100M	6278	5274	**5129**	1.22	12331	13234	**9855**	1.25	11153	8896	**8794**	1.27
0	1G	873	585	**546**	1.60	2427	1814	**1011**	2.40	2184	995	**897**	2.43
0	10G	335	129	**93**	3.60	440	331	**131**	3.36	501	196	**171**	2.93
0.2	100M	7855	5292	**5136**	1.53	18192	14632	**9860**	1.85	16517	8911	**8801**	1.88
0.2	1G	1134	682	**544**	2.08	2435	2506	**1024**	2.38	2381	996	**900**	2.65
0.2	10G	521	163	**92**	5.66	436	331	**129**	3.38	495	210	**171**	2.89
1	100M	9005	5751	**5130**	1.76	22330	22023	**9875**	2.26	21278	8948	**8803**	2.42
1	1G	2286	820	**554**	4.13	2464	2569	**1046**	2.36	2412	1049	**915**	2.64
1	10G	549	249	**99**	5.55	478	357	**140**	3.41	527	251	**170**	3.10
2	100M	10464	5679	**5147**	2.03	22354	24372	**9899**	2.26	21330	9475	**8824**	2.42
2	1G	2982	1028	**561**	5.32	2496	2647	**1073**	2.33	2447	1088	**934**	2.62
2	10G	627	338	**109**	5.75	533	437	**214**	2.49	579	324	**179**	3.23

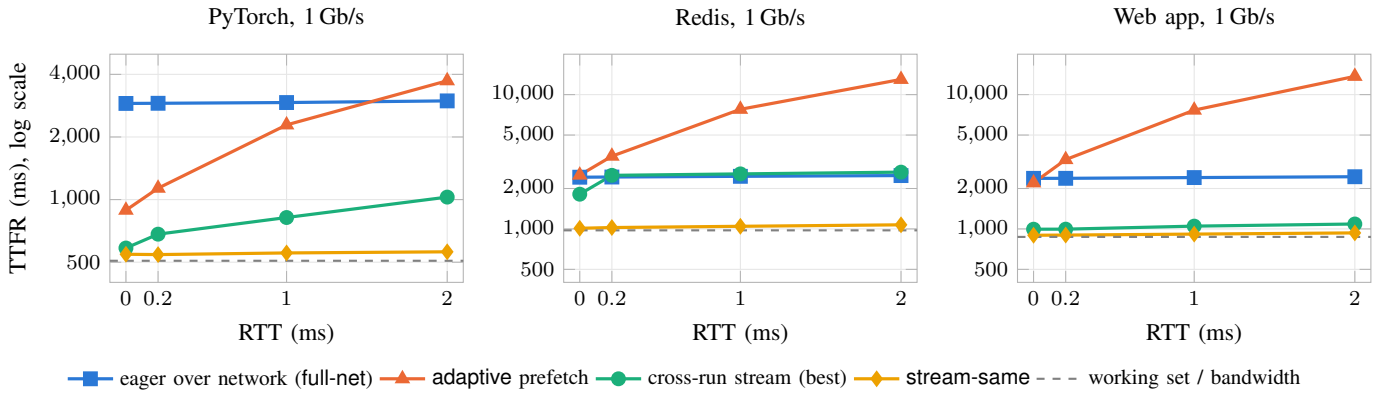


Fig. 1. TTFR against RTT at 1 Gb/s on x86. Lazy prefetching grows by about one RTT per remaining fault; eager pays for the whole image once; same-image streaming stays within 11 % of the working set's transfer time at every RTT.

web app's two recordings overlap 99.7 %, and plain stream wins all 12 cells by 1.25–2.56×. For Redis only 64.5 % of one recording's pages appear in the other. A stream mode beats the best existing policy by 1.22–1.34× in 6 cells, ties within 2 % in one, and loses the other five by 3–9 %: thousands of misses at an RTT or more each eat the gain.

Same-image trace. stream-same wins all 36 cells: by 1.22–5.75× over the best existing policy for PyTorch, 1.25–3.41× for Redis and 1.27–3.23× for the web app, and by 1.01–3.10× over the best cross-run stream. Demand faults fall to 0–4 misses for PyTorch and a median under 120 faults of any class for Redis. TTFR becomes nearly flat in RTT (PyTorch at 1 Gb/s: 546 to 561 ms over 0–2 ms) and tracks bandwidth. It lands within 1–2 % of working set/bandwidth at 100 Mb/s, 3–11 % at 1 Gb/s, and 30–120 ms above it at 10 Gb/s, where restore overheads dominate a 50–100 ms transfer.

The cost moved to the record pass. The record pass is a demand-only lazy restore. For PyTorch it takes 0.8–6.3 s at zero RTT and 32–38 s at 2 ms; for Redis 65–78 s at 2 ms. stream-same is therefore a number for the second and later restores of a snapshot, not the first. Systems that restore one snapshot many times, such as serverless platforms and agent sandboxes, amortize it; a one-off migration cannot, and must use a cross-run trace or an existing policy.

VII. WHY CROSS-RUN TRACES MISS

Every miss costs at least one RTT on the demand connection, and more when its reply queues behind the stream at a slow bottleneck (§VIII), so the trace's coverage of the next run decides how close a cross-run stream gets to stream-same.

PyTorch: fresh heap. Pairs of x86 PyTorch recordings share 94–98.6 % of pages (97.6–98.6 % for the pairs behind Table II). The differences sit in the `brk` heap: in one pair, one run had a 617-page heap region the other lacked. Every run allocates some fresh heap that no earlier trace can name.

Web app: reproducible. Python randomizes string hashes per process, which changes dictionary layouts, yet two web-app recordings share 99.7 % of pages on x86 and 99.9 % on the Pi. Cross-run stream is within 1–17 % of stream-same at 100 Mb/s and 1 Gb/s.

Redis on x86: layout jitter. The two x86 Redis traces have the same number of pages in each 256 MB region (27,210 vs 27,212), but the long runs of pages sit a few pages apart: a 1313-page run starts at `...2f8e000` in one trace and `...2f7b000` in the other. The process maps are identical

under `setarch -R`; placement inside the allocator's arenas differs. We started the server three times with ASLR off, loaded the same 200k keys, and read the addresses of 201 sampled keys with `DEBUG OBJECT`. Ranges span repeated sessions; these ad-hoc scripts are not part of the artifact:

setup	keys at the same address
x86, Valkey 9.1.2, jemalloc 5.4.0	64–183 / 201
hash seed fixed	154–182 / 201
jemalloc 5.3.1 swapped in	142–156 / 201
pinned to 4 CPUs / 1 CPU	65–135 / 25–146
Pi, Valkey 8.1.1, jemalloc 5.3.0	197–201 / 201
Pi, Redis 8.0.2	193–199 / 201

The mismatches are whole-page offsets (4, 8 or 20 KB) that appear and vanish along the insertion order rather than accumulating, consistent with freed multi-page extents being reused in a timing-dependent order. Valkey's random hash seed contributes (its hash table allocates overflow buckets depending on where keys hash), and the allocator build contributes; disabling active rehashing or jemalloc's decay-based purging does not help. We did not isolate the rest, nor separate Valkey 9.1 from 8.1. The same server family is reproducible on the Pi, where the cross-run trace overlaps 99.2 % and cross-run streaming comes within 5 % of stream-same (§IX).

Implication. Cross-run traces work only for programs whose heap layout is reproducible, and that is a property of the build and platform, not just of the program. A same-image trace is immune: its addresses come from the image itself.

VIII. Streaming the Rest of the Image

stream-all covers misses by streaming every remaining page after the trace. It cuts PyTorch faults to about 50 and wins at 10 Gb/s with high RTT, but on loopback it loses badly at 100 Mb/s: in one run 55 misses cost 12.4 s, about 225 ms each, because each demand reply queues behind the stream at the shaped bottleneck. In a three-iteration side study at 0 ms/100 Mb/s, stream-all adds 6.5 s over stream with CUBIC; delay-based congestion control (Vegas) on the stream socket cuts that to 2.2 s, and pacing the rest at 0.7× the measured delivery rate to 0.4 s; TCP-LP does nothing, because demand traffic is too small to count as a competing flow. No endpoint-only variant matches plain stream at 100 Mb/s.

On the Pis, the same comparison shows a penalty of about 1.1 ms per miss, not 225 ms, and it is unchanged when Pi B's root qdisc is switched from `fq_codel` to a plain `pfifo` (Table III). We believe TCP Small Queues bounds how much of the sender's data waits in the qdisc and driver, so the queue a demand reply waits behind is a millisecond or two at line rate, whereas `netem` on loopback holds a deep emulated queue. We did not verify the TSQ explanation directly. The loopback FIFO penalty should be read as an emulation artifact, not an estimate for a real link.

IX. Real Hardware: Two Raspberry Pi 5s

Table IV repeats the comparison on two Raspberry Pi 5s.

The loopback result holds. For PyTorch, cross-run stream-pf wins every delay by 1.46–2.96×, close to the x86 1 Gb/s cells (1.49–2.90×), and stream-same by 1.82–4.78×, with 13–18 misses per run. The existing policies order the same way: prefetch wins at low RTT and eager from about 2 ms. Each cross-run miss costs about one RTT plus 0.3–0.7 ms on the Pi; prefetching around misses removes nearly half of them (medians 548–581 to 299–309).

TABLE III
Pi PyTorch, stream and stream-all with Pi B's qdisc `fq_codel` vs `pfifo`: TTFR (ms) and median ms per miss.

mode	delay	fq_codel	pfifo	ms/miss fq	ms/miss pfifo
stream	0	728	718	0.88	0.91
stream	1000	1127	1071	1.58	1.62
stream	2000	1594	1405	2.48	2.54
stream-all	0	1145	1127	1.99	2.25
stream-all	1000	1249	1178	2.50	2.63
stream-all	2000	1372	1292	3.53	3.56

Redis and the web app gain 2.6–4.0×. Redis touches 111 MB of a 437 MB image before its first reply; eager moves all of it (about 3.5 s at line rate), streaming moves the working set. stream-same sits at the bandwidth floor: 111 MB at 1 Gb/s is about 0.9 s, it took 1.00–1.01 s, the process overtook the stream about 400 times ("ahead" faults) and only 7 pages missed. The web app touches 42 % of its image, so its gain is smaller (2.6–2.7×). On the Pi both have reproducible layouts (99.2 % and 99.9 % trace overlap), so cross-run streaming comes within 5 % of stream-same. For the web app, prefetching around misses *hurts* (stream-pf is 17–61 % slower than stream): it puts pages the process never needs on the link ahead of ones it does.

Old lazy policies collapse with RTT. Random access defeats sequential prefetch, which still leaves 6–9k faults for Redis and the web app: the adaptive prefetcher reaches 13.4–13.5 s at 2.16 ms RTT, more than three times the eager time.

X. After the First Response

TTFR rewards fetching exactly what the first request needs. A restored service keeps serving, though, and its next requests read pages the trace never named. We measured this on the Pis with the web app: right after the probe, the restored process serves 1000 further customer pages, disjoint from the probe's. Record runs stop at the first response, so traces do not include this load. Table V reports TTFR, the throughput of the follow-up phase, and when all 1200 requests have been served. Data: `pi-webapp-tput/`, `pi-webapp-tput-all2/`.

A trace alone wins the first response and loses the rest. After the stream ends, every page a later request touches is a demand fault at one RTT with nothing in flight. Follow-up throughput falls to 209–219 req/s at the base RTT and 21 req/s at 2 ms, 50× below eager; serving 1200 requests takes 48.6 s against 3.5 s for full-net.

Streaming the rest gets both. stream-all reaches its first response 2.3–2.5× sooner than eager and finishes all 1200 requests 11–13 % sooner, because the rest of the image arrives in the background (complete about 2.3 s in) while requests are served. Throughput while the rest is still arriving is about half

TABLE IV

PI TTFR (MS, MEDIAN OF 5) OVER A REAL 1 GBE LINK. COLUMNS ARE THE ADDED NETEM DELAY; RTT IS ABOUT 0.16 MS MORE. SPEEDUP: BEST STREAMING MODE OVER THE BEST EXISTING POLICY. IMAGE / TRACE: PYTORCH 306 / 61 MB, REDIS 437 / 111 MB, WEB APP 259 / 109 MB. PYTORCH ROWS USE CRIU 4.1.1 EXCEPT STREAM-SAME†, A LATER RUN WITH CRIU 4.2.1 IN WHICH FULL-NET AND STREAM-PF WERE WITHIN 6 % OF THESE VALUES. DATA: `PI-STREAM/`, `PI-PYTORCH-SAME/`, `PI-REDIS/`, `PI-WEBAPP/`.

	PyTorch			Redis 8.0.2			Web app		
mode	0	1 ms	2 ms	0	1 ms	2 ms	0	1 ms	2 ms
full (floor)	277	269	275	259	260	258	323	323	325
full-net	2939	2937	2947	4059	4064	4073	2601	2607	2607
lazy	2588	–	–	4586	–	–	4854	–	–
prefetch	1023	2850	5014	3606	9989	17709	2505	10396	18892
adaptive	1029	2569	4226	3616	8218	13433	2484	8008	13490
stream	728	1127	1594	1018	1155	1359	**966**	**973**	989
stream-pf	700	867	1076	1021	1049	1058	1127	1313	1591
stream-all	1145	1249	1372	1317	1715	1742	1322	1454	1526
stream-same	**561**†	**597**†	**617**†	**1002**	**1006**	**1014**	972	979	**985**
speedup	1.82×	4.30×	4.78×	3.60×	4.04×	4.02×	2.57×	2.68×	2.65×

TABLE V

PI WEB APP: TTFR, FOLLOW-UP THROUGHPUT (REQ/S) AND TIME UNTIL 1200 REQUESTS ARE SERVED (S), AT 0 AND 2 MS ADDED DELAY. MEDIAN OF 5.

	delay 0			delay 2 ms		
mode	TTFR	req/s	done	TTFR	req/s	done
full (local)	325	1071	1.26	324	1077	1.25
full-net	2597	1072	3.53	2610	1074	3.54
adaptive	2462	701	3.89	13406	237	17.6
stream	**965**	219	5.53	**980**	21	48.6
stream-pf	1019	414	3.43	1149	62	17.3
stream-same	970	209	5.76	983	21	48.6
stream-all	1045	497	**3.06**	1134	498	**3.14**

of eager's; once a process holds all its pages it runs at full speed (full, full-net: 1071–1077 req/s). Its TTFR here (1045 and 1134 ms) is lower than in the main Pi run of Table IV (1322 and 1526 ms), with under 2 % spread in each; we have not explained the difference.

A first attempt at this stream-all run recorded 5-page traces (both record passes finished in about 330 ms with 5 faults, as if restored eagerly); it started a minute after the previous run ended. A rerun two minutes later was normal. We discarded the first attempt and did not find the cause.

XI. IMPLEMENTATION PITFALLS

Four problems cost us time and are easy to repeat.

`UFFDIO_COPY` returns `EAGAIN` and leaves the thread asleep. While a `REMOVE` or `UNMAP` event is pending (jemalloc returns memory with `MADV_DONTNEED`), the kernel refuses `UFFDIO_COPY` and `UFFDIO_ZEROPAGE` with `EAGAIN` and does not wake the faulting thread. A handler that treats the error as "someone else will install it" hangs the process; our Redis runs stalled for hours. The fix is to issue `UFFDIO_WAKE` on that range after `EAGAIN` so the thread refaults.

Image formats drift. CRIU 4.2 writes the pagemap entry's page count as a new 64-bit field and leaves the old one zero; an older `pycriu` then reads every entry as empty and the page server serves zeros. We fall back to decoding the field by hand.

Restoring an image twice. stream-same restores one image twice. The first restored process appends to its stdout log, and CRIU refuses to restore a process whose open file has changed size; CRIU 4.2 also refuses to overwrite an existing pidfile. The harness truncates the log to its dump-time size and removes the pidfile between passes.

Nagle again. The web app's `http.server` writes headers and body separately. Over TCP, Nagle's algorithm holds the body until the client's delayed ACK: in a manual test the 200-request probe took 8.4 s instead of 120 ms. Production servers set `TCP_NODELAY`, and so does ours.

XII. DISCUSSION AND LIMITATIONS

What to deploy. If a snapshot is restored more than once, record its first-touch trace on the first restore. If only the first response matters, stream the trace alone: TTFR then approaches the working set's transfer time at every RTT we tested. If the process keeps serving, stream the rest of the image behind the trace, or every later request pays an RTT per cold page. Cross-run traces are worth trying only where the program's heap layout is reproducible; checking two recordings' overlap is cheap and predicts the outcome. Without a trace, the regime map applies: demand paging at low bandwidth and near-zero RTT, prefetching in a narrow middle for sequential workloads, and eager transfer once bandwidth × RTT is large or the access pattern is random.

Limitations.

- Three workloads, touching 19–42 % of their image before the first response. A workload that touches most of its image leaves streaming little to save over eager.
- x86 results use `netem` on loopback, with source and target on one host. The Pi results show one loopback effect (§VIII) does not transfer; others might not either.
- The page server is single-threaded Python. At 10 Gb/s its throughput and restore overheads, not the network, set the remaining 30–120 ms.
- Five iterations per cell. Cross-run streaming is noisy (standard deviation up to 56 % of the median); more

iterations would narrow its medians; all but two of the wins we report hold in every iteration.
- The side studies of congestion control and pacing (§VIII) use three iterations.
- Post-response behavior was measured for one workload, on the Pis.
- The cause of x86 heap-layout jitter is only partly identified, and three Pi anomalies are unexplained: the stream-all TTFR gap and the discarded run (§X), and one fast prefetch iteration (§III).

XIII. Conclusion

Against an honest eager baseline that pays for moving its image, lazy restore is not a free win: for the two random-access workloads, eager transfer beats every lazy policy at 9–10 of 12 network settings. What decides the map is one round trip per demand fault. Streaming a recorded first-touch trace by server push removes most of those round trips. With a trace recorded on the same snapshot it beats every existing policy at all 36 settings, by up to 5.75×, and stays within 11 % of the time to move the working set at 100 Mb/s and 1 Gb/s; on real hardware it cuts TTFR by up to 4.8×. A trace alone is not enough for a process that keeps serving: streaming the rest of the image behind it keeps most of the TTFR gain and serves the following load sooner than eager restore.

Artifact Availability

Code, harnesses and raw results are in the DistriProc repository (`src/`, `eval/`); each table names its result directories under `eval/results/`.

References

[1] "CRIU: Checkpoint/restore in userspace," https://criu.org, 2013, project documentation.

[2] "CRIU lazy migration," https://criu.org/Lazy_migration, 2017, design wiki.

[3] "userfaultfd(2) — Linux programmer's manual," https://man7.org/linux/man-pages/man2/userfaultfd.2.html, 2016.

[4] D. Ustiugov, P. Petrov, M. Kogias, E. Bugnion, and B. Grot, "Benchmarking, analysis, and optimization of serverless function snapshots," in *Proc. ASPLOS*, 2021, pp. 559–572.

[5] L. Ao, G. Porter, and G. M. Voelker, "FaaSnap: FaaS made fast using snapshot-based VMs," in *Proc. EuroSys*, 2022.

[6] B. Holmes, B. Dinis, L. Honcharuk, J. Fried, and A. Belay, "Rethinking process snapshots for near-warm serverless cold starts," in *Proc. USENIX OSDI*, 2026, earlier version: "Taming Serverless Cold Starts Through OS Co-Design", arXiv:2509.14292.

[7] "Under the hood: how AWS Lambda SnapStart optimizes function startup latency," https://aws.amazon.com/blogs/compute/under-the-hood-how-aws-lambda-snapstart-optimizes-function-startup-latency, aWS Compute Blog.

[8] M. Brooker, M. Danilov, C. Greenwood, and P. Piwonka, "On-demand container loading in AWS Lambda," in *Proc. USENIX ATC*, 2023.

[9] U. Maurya, "The prefetch paradox that wasn't: Four measurement artifacts in a CRIU lazy-restore study," 2026, companion report, under submission.

[10] M. R. Hines, U. Deshpande, and K. Gopalan, "Post-copy live migration of virtual machines," *ACM SIGOPS Operating Systems Review*, vol. 43, no. 3, pp. 14–26, 2009.

[11] "CRIU pull request #3200: image streaming for post-copy restore," https://github.com/checkpoint-restore/criu/pull/3200, open as of November 2026.

[12] X. Wei, F. Lu, T. Wang, J. Gu, Y. Yang, R. Chen, and H. Chen, "No provisioned concurrency: Fast RDMA-codesigned remote fork for serverless computing," in *Proc. USENIX OSDI*, 2023, pp. 497–517.

[13] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in *Proc. USENIX NSDI*, 2020.

[14] "Handling page faults on snapshot resume," https://github.com/firecracker-microvm/firecracker/blob/main/docs/snapshotting/handling-page-faults-on-snapshot-resume.md, firecracker documentation.

[15] H. Al Maruf and M. Chowdhury, "Effectively prefetching remote memory with Leap," in *Proc. USENIX ATC*, 2020.

[16] C. Branner-Augmon, N. Galstyan, S. Kumar, E. Amaro, A. Ousterhout, A. Panda, S. Ratnasamy, and S. Shenker, "3PO: Programmed far-memory prefetching for oblivious applications," arXiv:2207.07688, 2022.

[17] S. Hemminger, "Network emulation with NetEm," in *Linux Conf Au (linux.conf.au)*, 2005.